



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**

INFORME DE TRABAJO

Curso: Programación II

Docente: Ing. Breno Solim

Platero Maron, Victor Joseph

**Tacna – Perú
2026**

ÍNDICE

INTRODUCCIÓN	3
MARCO TEÓRICO	3
1. Definición y propósito:	3
2. Características destacadas:	4
3. Aplicaciones y casos de uso:	7
CONCLUSIÓN	9
REFERENCIAS BIBLIOGRÁFICAS	10

INTRODUCCIÓN

En el presente laboratorio se desarrolló una aplicación de escritorio utilizando el lenguaje de programación C# en el entorno Windows Forms bajo el Framework 4.8. El objetivo principal fue implementar un sistema de gestión de contactos personales aplicando los principios de Programación Orientada a Objetos (POO) y el uso de LINQ to SQL para la manipulación de datos.

La aplicación permite registrar, modificar, eliminar, buscar y visualizar contactos mediante una interfaz gráfica amigable y organizada. Además, se implementaron validaciones de datos y eventos en controles gráficos para garantizar un correcto funcionamiento del sistema.

Asimismo, el proyecto utiliza una base de datos SQL Server para almacenar la información de los contactos, estableciendo la conexión mediante LINQ to SQL y el archivo DataClasses1.dbml, permitiendo trabajar con objetos en lugar de consultas SQL complejas.

MARCO TEÓRICO

1. Definición y propósito:

La programación en C# dentro del entorno Windows Forms permite desarrollar aplicaciones de escritorio con interfaces gráficas dinámicas e interactivas. Este tipo de aplicaciones facilita la comunicación entre el usuario y el sistema mediante controles visuales que permiten ingresar datos, ejecutar procesos y mostrar resultados en tiempo real.

En el presente laboratorio se desarrolló un sistema de máquina expendedora

automatizada, el cual permite simular el funcionamiento de una máquina de ventas mediante la selección de productos a través de códigos numéricos y el ingreso de monedas válidas.

2. Características destacadas:

La programación en C# con Windows Forms permite desarrollar aplicaciones de escritorio con interfaces gráficas interactivas y fáciles de usar. Este tipo de aplicaciones facilita la entrada y manipulación de datos mediante controles visuales como botones, cuadros de texto, etiquetas y tablas.

En el presente laboratorio se implementó un sistema CRUD (Create, Read, Update, Delete), el cual permite administrar información de contactos de manera dinámica utilizando Programación Orientada a Objetos y LINQ.

Control	Propiedad	Valor
Form1	Text	Sistema de Contactos
Form1	StartPosition	CenterScreen
Form1	Size	900 x 550
lblTitle	Text	eContact
txtContactId	ReadOnly	True
txtFirstName	Placeholder	Ingrese nombre
txtLastName	Placeholder	Ingrese apellido
txtContactNo	Placeholder	Ingrese teléfono
txtAddress	Multiline	True
cmbGender	Items	Masculino, Femenino
btnAdd	Text	Agregar
btnUpdate	Text	Actualizar
btnDelete	Text	Eliminar
btnClear	Text	Limpiar
txtSearch	Text	Buscar

dgvContact	SelectionMode	FullRowSelect
dgvContact	AutoSizeColumnsMode	Fill
DataClasses1.dbml	Tecnología	LINQ to SQL
Tabla SQL	Nombre	tbl_contact

3. Aplicaciones y casos de uso:

e CONTACT
X

Contacto ID
 Nombre
 Apellido
 Contact No.
 Direccion
 Genero

Buscar

Agregar
Actualizar
Eliminar
Limpiar

Contactos
⌵

Propiedades

🔑
IdContacto

🔑
Nombre

🔑
Apellido

🔑
Telefono

🔑
Direccion

🔑
Genero

🔑
FechaRegistro

```

2
3 CREATE DATABASE BD_Contactos;
4 GO
5
6 USE BD_Contactos;
7 GO
8
9 -----
10 -- TABLA CONTACTOS
11 -----
12
13 CREATE TABLE Contactos
14 (
15     IdContacto INT PRIMARY KEY IDENTITY(1,1),
16     Nombre NVARCHAR(100) NOT NULL,
17     Apellido NVARCHAR(100) NOT NULL,
18     Telefono NVARCHAR(20) NOT NULL,
19     Direccion NVARCHAR(200) NULL,
20     Genero NVARCHAR(20) NOT NULL,
21     FechaRegistro DATETIME DEFAULT GETDATE()
22 );
23 GO
24
25 -----
26

```

% 1 0

Resultados		Mensajes					
	IdContacto	Nombre	Apellido	Telefono	Direccion	Genero	FechaRegistro
	1	Juan	Perez	987654321	Lima	Masculino	2026-05-11 20:24:54.847
	2	Maria	Lopez	912345678	Arequipa	Femenino	2026-05-11 20:24:54.847
	3	Carlos	Ramirez	956321478	Cusco	Masculino	2026-05-11 20:24:54.847

using System;

```

using System.Linq;
using System.Windows.Forms;

namespace Trabajo_Contact
{
    public partial class Form1 : Form
    {
        // Conexión a la base de datos mediante LINQ to SQL
        DataClasses1DataContext db = new DataClasses1DataContext();

        public Form1()
        {
            InitializeComponent();

            // Llenamos las opciones del cuadrado desplegable de género
            cmbGender.Items.Add("Masculino");
            cmbGender.Items.Add("Femenino");

            // Al abrir el programa, mostramos la lista de contactos
            MostrarContactos();
        }

        // Función para traer los datos de la base de datos y ponerlos en la tabla (Grid)
        private void MostrarContactos()
        {
            var lista = from c in db.Contactos
                select new
                {
                    c.IdContacto,
                    c.Nombre,
                    c.Apellido,
                    c.Telefono,
                }
        }
    }
}

```

```

        c.Direccion,
        c.Genero
    };

    dgvContact.DataSource = lista.ToList();
}

// Limpia todos los cuadros de texto para dejar el formulario vacío
private void Limpiar()
{
    txtContactId.Text = "";
    txtFirstName.Text = "";
    txtLastName.Text = "";
    txtContactNo.Text = "";
    txtAddress.Text = "";
    txtSearch.Text = "";
    cmbGender.SelectedIndex = -1; // Deselecciona el género

    txtFirstName.Focus(); // Pone el cursor listo para escribir en el nombre
}

// Revisa que no falten datos importantes antes de guardar
private bool Validar()
{
    if (txtFirstName.Text.Trim() == "")
    {
        MessageBox.Show("Ingrese nombre");
        return false;
    }

    if (txtLastName.Text.Trim() == "")
    {
        MessageBox.Show("Ingrese apellido");
    }
}

```



```

        return false;
    }

    if (txtContactNo.Text.Trim() == "")
    {
        MessageBox.Show("Ingrese teléfono");
        return false;
    }

    if (cmbGender.Text == "")
    {
        MessageBox.Show("Seleccione género");
        return false;
    }

    return true; // Si todo está bien, devuelve "verdadero"
}

// Botón para agregar un nuevo contacto
private void btnAdd_Click(object sender, EventArgs e)
{
    if (!Validar()) return; // Si falta algún dato, no hace nada

    try
    {
        // Creamos un nuevo objeto de contacto con lo que escribimos en la pantalla
        Contactos c = new Contactos();

        c.Nombre = txtFirstName.Text;
        c.Apellido = txtLastName.Text;
        c.Telefono = txtContactNo.Text;
        c.Direccion = txtAddress.Text;
        c.Genero = cmbGender.Text;

        db.Contactos.InsertOnSubmit(c); // Lo preparamos para insertar
    }
    catch { }
}

```

```

        db.SubmitChanges(); // Guardamos en la base de datos real

        MessageBox.Show("Contacto agregado");

        MostrarContactos(); // Actualizamos la tabla

        Limpiar(); // Vaciamos los campos
    }

    catch (Exception ex)
    {
        MessageBox.Show(ex.Message); // Por si ocurre algún error inesperado
    }
}

// Botón para modificar un contacto existente
private void btnUpdate_Click(object sender, EventArgs e)
{
    if (txtContactId.Text == "")
    {
        MessageBox.Show("Seleccione un contacto");

        return;
    }

    try
    {
        int id = Convert.ToInt32(txtContactId.Text);

        // Buscamos el contacto en la base de datos usando su ID
        var contacto = (from c in db.Contactos
                        where c.IdContacto == id
                        select c).FirstOrDefault();

        if (contacto != null)
        {

```

```

// Cambiamos los datos viejos por los nuevos que están en los cuadros de texto
contacto.Nombre = txtFirstName.Text;

contacto.Apellido = txtLastName.Text;

contacto.Telefono = txtContactNo.Text;

contacto.Direccion = txtAddress.Text;

contacto.Genero = cmbGender.Text;

db.SubmitChanges(); // Guardamos los cambios

MessageBox.Show("Contacto actualizado");

MostrarContactos();

Limpiar();
}
}
catch (Exception ex)
{
    MessageBox.Show(ex.Message);
}
}

// Botón para borrar un contacto
private void btnDelete_Click(object sender, EventArgs e)
{
    if (txtContactId.Text == "")
    {
        MessageBox.Show("Seleccione un contacto");
        return;
    }

    // Preguntamos al usuario si está seguro para evitar borrados por accidente
    DialogResult r = MessageBox.Show("¿Desea eliminar el contacto?", "Confirmar",
    MessageBoxButtons.YesNo, MessageBoxIcon.Question);

```

```

if (r == DialogResult.Yes)
{
    try
    {
        int id = Convert.ToInt32(txtContactId.Text);

        var contacto = (from c in db.Contactos
                        where c.IdContacto == id
                        select c).FirstOrDefault();

        if (contacto != null)
        {
            db.Contactos.DeleteOnSubmit(contacto); // Borramos el registro
            db.SubmitChanges(); // Confirmamos el borrado en la base de datos

            MessageBox.Show("Contacto eliminado");

            MostrarContactos();

            Limpiar();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}

// Buscador: Se activa cada vez que escribimos una letra en el cuadro de búsqueda
private void txtSearch_TextChanged(object sender, EventArgs e)
{
    var buscar = from c in db.Contactos
                 where c.Nombre.Contains(txtSearch.Text)

```

```

        || c.Apellido.Contains(txtSearch.Text)

        || c.Telefono.Contains(txtSearch.Text)

select new
{
    c.IdContacto,
    c.Nombre,
    c.Apellido,
    c.Telefono,
    c.Direccion,
    c.Genero
};

dgvContact.DataSource = buscar.ToList(); // Muestra solo los resultados que coincidan
}

// Botón para limpiar manualmente el formulario
private void btnClear_Click(object sender, EventArgs e)
{
    Limpiar();
}

// Al hacer clic en un contacto de la tabla, pasamos sus datos a los cuadros de texto arriba
private void dgvContact_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow fila = dgvContact.Rows[e.RowIndex];

        txtContactId.Text = fila.Cells["IdContacto"].Value.ToString();
        txtFirstName.Text = fila.Cells["Nombre"].Value.ToString();
        txtLastName.Text = fila.Cells["Apellido"].Value.ToString();
        txtContactNo.Text = fila.Cells["Telefono"].Value.ToString();
    }
}

```

```
txtAddress.Text = fila.Cells["Direccion"].Value.ToString();  
cmbGender.Text = fila.Cells["Genero"].Value.ToString();  
}  
}  
}  
}
```

CONCLUSIÓN

Durante el desarrollo del presente laboratorio se logró implementar una aplicación funcional de gestión de contactos utilizando C#, Windows Forms, SQL Server y LINQ to SQL.

Se aplicaron conceptos fundamentales como Programación Orientada a Objetos, manipulación de eventos, validación de datos y consultas mediante LINQ. Además, se desarrolló una interfaz gráfica organizada y fácil de usar que mejora la interacción del usuario con el sistema.

En conclusión, este laboratorio permitió fortalecer conocimientos relacionados con el desarrollo de aplicaciones de escritorio, acceso a bases de datos y uso de tecnologías modernas para la administración de información de manera eficiente.

REFERENCIAS BIBLIOGRÁFICAS

Microsoft. (2024). Introducción a Windows Forms. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>

Microsoft. (2024). Eventos en C#. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/events/>

Microsoft. (2024). Controles en Windows Forms. Microsoft Learn.

<https://learn.microsoft.com/es-es/dotnet/desktop/winforms/controls/>